

ADR-001 — Emplacement du contrôle d'accès aux données de santé

Statut : Accepté (périmètre POC) · **Date :** 01/07/2026 · **Décideurs :** Analyste technique (Hervé), équipe projet MediPlan · **Contexte :** POC du risque critique EBIOS R1

Contexte

MediPlan traite des données de santé à caractère personnel (hébergement HDS). L'analyse EBIOS a retenu **R1 — fuite de données de santé** comme risque critique, avec **R2 — compromission d'un compte** en risque élevé. Le POC doit prouver la faisabilité de la mitigation : **chiffrement des données sensibles au repos** (mesure principale contre R1) et **contrôle d'accès par rôle (RBAC)** (mesure complémentaire, surtout contre R2).

Il faut décider où s'applique le RBAC. Forces en présence : stack cible NestJS + PostgreSQL (CDC) ; périmètre **spike** scolaire (minimal, démontrable) ; environnement de POC **sans serveur PostgreSQL** disponible ; nécessité de convaincre en soutenance.

Décision

Pour le POC, le RBAC est implémenté au **niveau applicatif** (à la manière des *Guards* NestJS). Le chiffrement reste applicatif avec la **clé conservée hors base**. **PostgreSQL RLS (Row-Level Security)** est retenu comme **couche de durcissement recommandée en production**, non implémentée dans le POC.

Options considérées

Option A — RBAC applicatif (Guards NestJS)

Dimension	Évaluation
Complexité	Faible
Coût	Faible
Robustesse	Moyenne (protection au niveau applicatif uniquement)
Familiarité équipe	Élevée — natif NestJS
Exécutable dans le POC	Oui

Pour : cohérent avec la stack du CDC ; simple et rapide ; testable et démontrable de bout en bout dans l'environnement actuel ; logique centralisable dans un *Guard* réutilisable.

Contre : protection contournée si un attaquant accède *directement* à la base (défense en profondeur incomplète) ; dépend de la rigueur du code (chaque point d'accès doit appliquer le contrôle) ; aucune garantie au niveau de la donnée elle-même.

Option B — PostgreSQL RLS (Row-Level Security)

Dimension	Évaluation
Complexité	Élevée
Coût	Moyen
Robustesse	Élevée (contrôle au plus près de la donnée)
Familiarité équipe	Faible

Pour : la règle d'accès vit dans la base ; elle tient *même si l'application est contournée* ; déclarative et centralisée ; véritable défense en profondeur.

Contre : complexité et courbe d'apprentissage ; couplage fort au SGBD ; nécessite de mapper l'identité applicative vers la session PostgreSQL (`current_setting` / JWT) — non trivial ; non exécutable dans l'environnement de POC ; débogage plus opaque.

Option C — Les deux (défense en profondeur)

Combinaison applicatif + RLS. C'est la **cible de production**, mais surdimensionnée et non exécutable pour un spike.

Analyse des compromis

Le POC vise à **prouver une faisabilité** et à **convaincre en soutenance**, en temps court et sans serveur PostgreSQL. L'option A maximise la démontrabilité et la cohérence avec la stack, au prix d'une robustesse moindre. L'option B maximise la robustesse mais n'est ni exécutable ici ni proportionnée au périmètre.

Point clé : la mesure *centrale* contre R1 reste le **chiffrement au repos avec clé hors base** — même sans RLS, un dump volé demeure illisible. Le RBAC, quel que soit son niveau, est une mesure *complémentaire* qui adresse surtout R2 (limiter ce qu'un compte compromis peut lire). Reporter le RLS ne laisse donc pas R1 découvert.

Conséquences

- **Plus facile :** livrer et démontrer le POC ; rester aligné au CDC ; itérer rapidement.
- **Plus difficile / dette :** le POC ne prouve pas la couche RLS ; il faudra l'ajouter en production pour la défense en profondeur.
- **Risque résiduel :** un accès direct à la base contourne le RBAC applicatif — atténué par le chiffrement au repos et par le moindre privilège sur les comptes PostgreSQL.
- **À revisiter :** en phase de réalisation, implémenter RLS + moindre privilège des rôles DB, et réévaluer selon les exigences HDS.

Actions

- ☐ Implémenter le spike Node/TypeScript : chiffrement AES-256-GCM (clé via variable d'environnement) + Guard de rôle.
- ☐ Démontrer : donnée illisible en base, déchiffrement avec la clé, refus d'accès pour un rôle non autorisé.
- ☐ Documenter dans `poc/README.md` le lien EBIOS (R1 / R2) et la recommandation RLS en production.
- ☐ En réalisation : ajouter PostgreSQL RLS + moindre privilège des rôles DB (défense en profondeur).

En résumé : **RBAC applicatif pour le POC** (démontrable, cohérent stack), le **chiffrement au repos** portant la charge principale contre R1, et le **RLS PostgreSQL** identifié comme durcissement de production. Décision proportionnée au périmètre du spike, sans laisser le risque critique découvert.